

UNIVERSIDADE FEDERAL DE MATO GROSSO  
CAMPUS ARAGUAIA  
INSTITUTO DE CIÊNCIAS EXATAS E DA TERRA – ICET  
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Luiz Henrique Ribeiro Torres  
Raissa Caroline Cavalcanti da Silva

**OPZIP: ZIP PARALELO COM ESCRITA ORDENADA**

*Sincronização produtor-consumidor com pthreads aplicada a compressão RLE*

Prof. Dr. Maxweel Silva Carmo

Barra do Garças, Mato Grosso  
Julho de 2026

## 1 INTRODUÇÃO

Este relatório descreve a implementação do opzip, uma variante paralela de compressão run-length encoding (RLE) baseada em pthreads. O trabalho tem como objetivo aprofundar o uso de threads POSIX — em particular mutexes e condition variables — na construção de um pipeline produtor-consumidor real, no qual blocos comprimidos são escritos no arquivo de saída assim que ficam prontos, respeitando a ordem original do arquivo, por meio de um buffer circular limitado compartilhado entre threads produtoras e uma única thread consumidora (escritora). Diferentemente do modelo fork-join clássico (em que cada thread comprime seu pedaço isoladamente e os resultados são concatenados ao final), o opzip mantém sincronização contínua durante toda a execução, o que exige um cuidado adicional de projeto para equilibrar paralelismo e ordem de escrita.

## 2 FUNDAMENTAÇÃO TEÓRICA E RELAÇÃO COM SISTEMAS OPERACIONAIS

O opzip é, em essência, um exercício aplicado de três conceitos centrais estudados na disciplina de Sistemas Operacionais: threads como unidade de execução concorrente, exclusão mútua sobre regiões críticas compartilhadas, e sincronização condicional entre threads que dependem do progresso umas das outras. A seguir, cada um desses conceitos é relacionado diretamente às decisões de projeto do código.

### 2.1 Threads como abstração de execução concorrente

Uma thread é uma unidade de execução que compartilha o mesmo espaço de endereçamento de um processo, mas mantém seu próprio contexto de execução (pilha, registradores, contador de programa). O sistema operacional é responsável por escalonar essas threads entre os núcleos disponíveis, alternando o contexto de execução de forma transparente ao programador. No opzip, essa abstração é explorada criando N threads produtoras que competem pelo mesmo conjunto de blocos de trabalho, e uma thread consumidora dedicada. Como as threads compartilham memória (o buffer de entrada, o índice de próximo bloco e o buffer circular de saída), qualquer acesso concorrente a essas estruturas exige sincronização explícita — sem ela, o sistema operacional pode intercalar as operações de threads distintas de forma imprevisível, produzindo uma condição de corrida (race condition).

### 2.2 Exclusão mútua com mutexes

Um mutex (mutual exclusion lock) garante que, em um dado instante, no máximo uma thread execute a seção de código protegida por ele. No opzip, dois mutexes distintos protegem duas regiões críticas independentes: `mutex_bloco` protege apenas a leitura e o

incremento do contador `proximo_bloco` (uma operação curtíssima), enquanto `mutex_buffer` protege o acesso às  $K$  posições do buffer circular compartilhado entre produtoras e consumidora. Separar essas duas regiões críticas em mutexes distintos evita que a distribuição de blocos (rápida) fique bloqueada esperando por operações de buffer (potencialmente mais lentas), reduzindo a contenção desnecessária entre partes não relacionadas do algoritmo.

### 2.3 O problema do produtor-consumidor e o buffer limitado

O opzip é uma instanciação direta do clássico problema do produtor-consumidor com buffer limitado (bounded buffer problem), amplamente discutido no capítulo de Condition Variables do OSTEP. Nesse problema, threads produtoras geram itens que devem ser consumidos por outra thread, mas o buffer compartilhado tem capacidade finita: produtoras devem bloquear quando o buffer está cheio, e a consumidora deve bloquear quando o buffer está vazio. A solução clássica usa duas condition variables (uma para cada direção de espera) e um mutex protegendo o estado compartilhado. O opzip segue exatamente esse padrão, com uma complicação adicional em relação à versão didática do OSTEP: os itens produzidos (blocos comprimidos) precisam ser consumidos (escritos) em uma ordem específica — a ordem original do arquivo — e não na ordem em que ficam prontos. Essa restrição de ordenação transforma o buffer circular simples em uma janela deslizante sobre uma sequência ordenada de blocos, detalhada na Seção 4.

### 2.4 Condition variables e a eliminação de busy-waiting

Uma alternativa ingênua para uma thread esperar por uma condição seria testar essa condição em um laço (busy-waiting ou polling), consumindo ciclos de CPU continuamente mesmo sem realizar trabalho útil. Além de desperdiçar CPU que poderia ser escalonada para outras threads ou processos, esse padrão pode causar inversão de prioridade e degradação de desempenho sob carga. O opzip evita esse problema utilizando `pthread_cond_wait` exclusivamente: uma thread que precisa esperar por uma condição (espaço livre no buffer, ou dados prontos) libera atomicamente o mutex e suspende sua execução, sendo devolvida à fila de prontos pelo escalonador do sistema operacional somente quando outra thread sinalizar a condition variable correspondente. Isso foi verificado experimentalmente: mesmo em cenários de forte serialização ( $K = 1$ , testado durante o desenvolvimento), o tempo de CPU consumido pelas threads bloqueadas permaneceu próximo de zero, confirmando a ausência de busy-waiting.

### 2.5 Race conditions e deadlock

Dois riscos clássicos de programação concorrente guiaram o projeto e os testes do opzip. Uma race condition ocorre quando o resultado de um programa depende do entrelaçamento (interleaving) específico entre as instruções de threads distintas acessando

um mesmo dado compartilhado sem sincronização adequada; no opzip, isso foi verificado ausente por meio do ThreadSanitizer (compilação com `-fsanitize=thread`), que instrumenta o binário para detectar acessos concorrentes não sincronizados a memória compartilhada. Um deadlock ocorre quando duas ou mais threads ficam bloqueadas indefinidamente, cada uma esperando por um recurso que a outra jamais liberará. Durante o desenvolvimento do opzip, um bug real desse tipo foi identificado e corrigido — descrito em detalhe na Seção 4.3 — no qual uma condição de espera insuficientemente restritiva permitia que um bloco de índice alto ocupasse prematuramente uma posição do buffer, travando a consumidora para sempre à espera de um bloco de índice mais baixo que nunca conseguiria ser escrito naquela posição.

### 3 ARQUITETURA DO SISTEMA

A Figura 1 (descritiva) resume o fluxo de dados do opzip. A thread principal lê o arquivo de entrada inteiro para memória em uma única leitura sequencial (sem concorrência nessa etapa) e calcula  $\text{num\_blocos} = \text{ceil}(\text{tamanho\_arquivo} / \text{TAM\_BLOCO})$ . Um índice compartilhado, `proximo_bloco`, protegido por mutex, distribui os blocos dinamicamente entre as threads produtoras: cada thread retira o próximo índice disponível assim que termina o bloco anterior, o que balanceia automaticamente a carga entre threads mais rápidas e mais lentas — uma thread que recebeu um bloco fácil de comprimir naturalmente pega o próximo bloco disponível mais cedo, sem que nenhuma atribuição estática de blocos por thread seja necessária.

A comunicação entre as produtoras e a consumidora é feita por um buffer circular de  $K$  posições. A solução entregue (opzip.c) usa um único mutex global (`mutex_buffer`) protegendo o buffer inteiro, com duas condition variables: `cond_slot_livre`, na qual as produtoras aguardam espaço, e `cond_slot_pronto`, na qual a consumidora aguarda dados. Nenhuma espera é feita por polling; toda sincronização utiliza `pthread_cond_wait`, conforme discutido na Seção 2.4.

## 4 DETALHAMENTO DO CÓDIGO

### 4.1 Estrutura de dados do buffer circular

Cada posição do buffer circular é representada pela struct `Slot`, contendo o índice original do bloco (necessário para a consumidora validar se o dado presente é realmente o próximo a ser escrito), o ponteiro para os dados já comprimidos, o tamanho desses dados e uma flag pronto indicando se a posição contém dados válidos ainda não consumidos:

```
typedef struct {
    int indice_bloco;
    char *dados_comprimidos;
    size_t tamanho_comprimido;
    int pronto;
} Slot;
```

## 4.2 Compressão run-length encoding

A função `comprimir_bloco` implementa o RLE propriamente dito. Ela percorre o bloco contando repetições consecutivas do mesmo caractere e grava cada run como um registro de 5 bytes (4 bytes de contagem em `int32_t` seguidos de 1 byte de caractere), usando `memcpy` em vez de escrever a struct diretamente — evitando que o compilador insira padding e quebre o formato de 5 bytes exigido pelo enunciado:

```
static char *comprimir_bloco(const char *dados, size_t tamanho,
                             size_t *tamanho_saida) {
    size_t capacidade = (tamanho == 0) ? 1 : tamanho * 5;
    char *saida_buf = malloc(capacidade);
    size_t pos = 0, i = 0;
    while (i < tamanho) {
        char c = dados[i];
        size_t count = 1;
        while (i + count < tamanho && dados[i + count] == c) {
            count++;
        }
        int32_t cnt = (int32_t) count;
        memcpy(saida_buf + pos, &cnt, sizeof(int32_t));
        pos += sizeof(int32_t);
        saida_buf[pos] = c;
        pos += 1;
        i += count;
    }
    *tamanho_saida = pos;
    return saida_buf;
}
```

O buffer de saída é alocado com o tamanho de pior caso (`tamanho x 5`), correspondente a um arquivo sem nenhum caractere repetido, em que cada byte de entrada vira uma run individual de 5 bytes de saída.

## 4.3 Thread produtora e a janela deslizando

Cada thread produtora executa um laço que retira o próximo índice de bloco disponível, comprime esse bloco (fora de qualquer região crítica, para não bloquear outras threads durante o trabalho pesado de CPU) e, em seguida, disputa o mutex do buffer apenas para inserir o resultado já pronto:

```

pthread_mutex_lock(&mutex_buffer);
while (idx >= proximo_a_escrever + K) {
    pthread_cond_wait(&cond_slot_livre, &mutex_buffer);
}
buffer[posicao].indice_bloco = idx;
buffer[posicao].dados_comprimidos = comp;
buffer[posicao].tamanho_comprimido = tamanho_comp;
buffer[posicao].pronto = 1;
pthread_cond_broadcast(&cond_slot_pronto);
pthread_mutex_unlock(&mutex_buffer);

```

A condição de espera  $\text{idx} \geq \text{proximo\_a\_escrever} + K$  é o ponto mais delicado de todo o projeto, e vale a pena detalhar por que ela é necessária em vez de simplesmente checar se  $\text{buffer}[\text{posicao}].\text{pronto}$  é falso. Como os blocos são distribuídos dinamicamente (Seção 3), não há garantia de que os blocos terminem de ser comprimidos na mesma ordem em que foram distribuídos: uma thread pode terminar de comprimir um bloco de índice alto (por exemplo,  $\text{idx} = 10$ , com  $K = 8$ , ocupando a posição  $10 \bmod 8 = 2$ ) antes de outra thread terminar o bloco de índice mais baixo que ocupa a mesma posição do buffer ( $\text{idx} = 2$ ). Se a condição de espera checasse apenas  $\text{pronto}$ , a posição 2 apareceria como “livre” (pois o bloco 2 sequer começou a ser escrito ali) e o bloco 10 seria inserido prematuramente. Quando o bloco 2 finalmente terminasse de ser comprimido, sua thread encontraria a posição 2 ocupada pelo bloco 10 e ficaria bloqueada esperando ela ser liberada — mas a consumidora, por sua vez, está esperando exatamente pelo bloco 2 naquela posição, e nunca o receberá, resultando em deadlock permanente. A condição correta, baseada na janela deslizante  $[\text{proximo\_a\_escrever}, \text{proximo\_a\_escrever} + K)$ , impede que qualquer bloco além dessa janela seja inserido, garantindo que uma posição só seja reutilizada depois que o bloco anterior que a ocupava (índice  $K$  posições atrás) já tenha sido efetivamente escrito pela consumidora. Esse bug foi encontrado durante os testes de correção do projeto, ao observar que arquivos com mais de aproximadamente  $2K$  blocos travavam indefinidamente com a implementação ingênua, e corrigido antes da entrega final.

#### 4.4 Thread consumidora

A thread consumidora mantém um contador  $\text{proximo\_a\_escrever}$  e, para cada bloco, espera até que a posição correspondente contenha exatamente o índice esperado (não basta a posição estar pronta — ela precisa conter o bloco certo, já que, em tese, outra thread poderia ter inserido ali um bloco diferente caso a janela fosse maior que  $K$ ):

```

pthread_mutex_lock(&mutex_buffer);
while (!(buffer[posicao].pronto &&
        buffer[posicao].indice_bloco == proximo_a_escrever)) {
    pthread_cond_wait(&cond_slot_pronto, &mutex_buffer);
}
char *dados = buffer[posicao].dados_comprimidos;
size_t tamanho = buffer[posicao].tamanho_comprimido;
pthread_mutex_unlock(&mutex_buffer);

fwrite(dados, 1, tamanho, saida);
free(dados);

pthread_mutex_lock(&mutex_buffer);
buffer[posicao].pronto = 0;
proximo_a_escrever++;
pthread_cond_broadcast(&cond_slot_livre);
pthread_mutex_unlock(&mutex_buffer);

```

Note que a escrita em disco (fwrite) é feita fora da região crítica, liberando o mutex antes de uma operação de I/O potencialmente lenta — assim, threads produtoras podem continuar inserindo blocos no buffer enquanto a consumidora escreve o bloco anterior no arquivo de saída.

#### 4.5 Distribuição dinâmica de blocos e encerramento

A distribuição de blocos usa um mutex separado (mutex\_bloco), independente do mutex do buffer, protegendo apenas o contador proximo\_bloco:

```

pthread_mutex_lock(&mutex_bloco);
if (proximo_bloco >= num_blocos) {
    pthread_mutex_unlock(&mutex_bloco);
    break;
}
int idx = proximo_bloco;
proximo_bloco++;
pthread_mutex_unlock(&mutex_bloco);

```

Quando proximo\_bloco atinge num\_blocos, cada thread produtora encerra seu laço naturalmente e retorna. A thread principal então executa pthread\_join em todas as produtoras e, por fim, na consumidora (que só termina seu próprio laço quando proximo\_a\_escrever alcança num\_blocos), garantindo um encerramento limpo sem loops infinitos ou threads órfãs. Toda memória alocada dinamicamente (buffer de entrada, buffer circular, vetor de identificadores de threads, e os dados comprimidos de cada slot) é liberada antes do programa terminar.

## 5 ESCOLHAS DE TAM\_BLOCO E K

O TAM\_BLOCO padrão adotado foi de 1 MB, independente do número de threads, conforme exigido pelo enunciado. Blocos muito pequenos aumentam o overhead de sincronização (mais operações de lock e unlock por byte processado) e degradam a taxa de compressão nas bordas dos blocos; blocos muito grandes reduzem o paralelismo disponível,

pois restam poucos blocos para muitas threads, além de aumentar a latência até a primeira escrita no arquivo de saída. O valor de 1 MB equilibra os dois efeitos para arquivos de dezenas de megabytes ou mais.

O valor padrão de  $K$  adotado foi 2 vezes o número de threads. Um  $K$  pequeno demais força as produtoras a aguardar com frequência por espaço no buffer, serializando o pipeline; um  $K$  grande demais consome mais memória sem benefício adicional, já que o gargalo real é a taxa de escrita da consumidora, única e sequencial. Na prática, um  $K$  equivalente ao dobro do número de threads dá folga suficiente para que threads mais rápidas não fiquem bloqueadas esperando a consumidora, sem desperdiçar memória.

## 6 AMBIENTE DE TESTES

Os experimentos de desempenho apresentados a seguir foram executados em um MacBook Pro com processador multi-núcleo, utilizando um arquivo de teste de aproximadamente 150 MB, com conteúdo parcialmente repetitivo gerado sinteticamente. Cada medição corresponde à mediana de três execuções. O script `benchmark.py`, incluído na entrega, automatiza a coleta dessas medidas e a geração dos gráficos. A correção funcional foi validada separadamente com arquivos vazios, menores que um bloco, múltiplos exatos do tamanho de bloco, altamente repetitivos e com dados aleatórios, comparando byte a byte a saída do descompressor de referência (`ounzip`) com o arquivo original, além de execuções com ThreadSanitizer sem nenhum aviso de data race.

## 7 SPEEDUP VERSUS NÚMERO DE THREADS

A Figura 2 apresenta o speedup medido ( $T_1/T_n$ ) em função do número de threads produtoras, comparado ao speedup ideal linear.



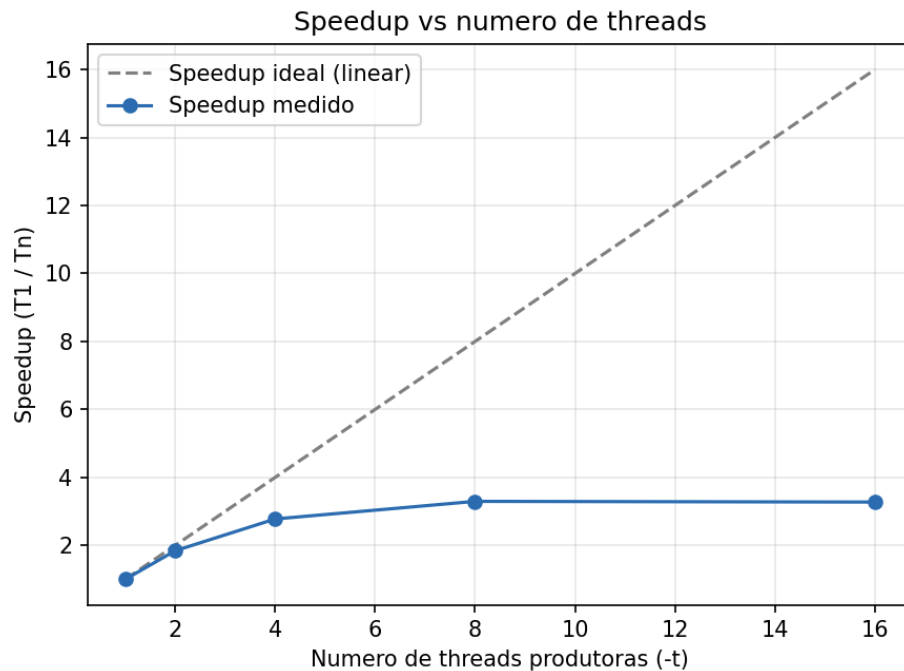


Figura 2 – Speedup medido vs. número de threads produtoras.

O speedup cresce de forma consistente até 8 threads (1,0x com 1 thread; 1,84x com 2; 2,78x com 4; 3,29x com 8), aproximando-se de um comportamento quase linear nessa faixa. A partir de 8 threads o ganho estagna (3,27x com 16 threads, praticamente idêntico ao valor com 8), indicando saturação do paralelismo disponível na máquina de teste. Dois fatores explicam esse platô: primeiro, o número de threads passou a exceder o número de núcleos físicos úteis para esta carga, de modo que threads adicionais competem por CPU em vez de acrescentar paralelismo real; segundo, a thread consumidora é única e escreve sequencialmente no arquivo de saída, tornando-se o gargalo estrutural do pipeline — acima de certo número de produtoras, a taxa de escrita da consumidora passa a limitar o desempenho total, independentemente de quantas threads comprimam blocos em paralelo. Esse comportamento ilustra a Lei de Amdahl aplicada ao pipeline: a fração sequencial do algoritmo (a escrita ordenada) limita o speedup máximo alcançável, independentemente de quantas threads produtoras sejam adicionadas.

## 8 EFEITO DO PARÂMETRO K NO DESEMPENHO

A Figura 3 apresenta o tempo de execução em função de K, com o número de threads fixo em 4.

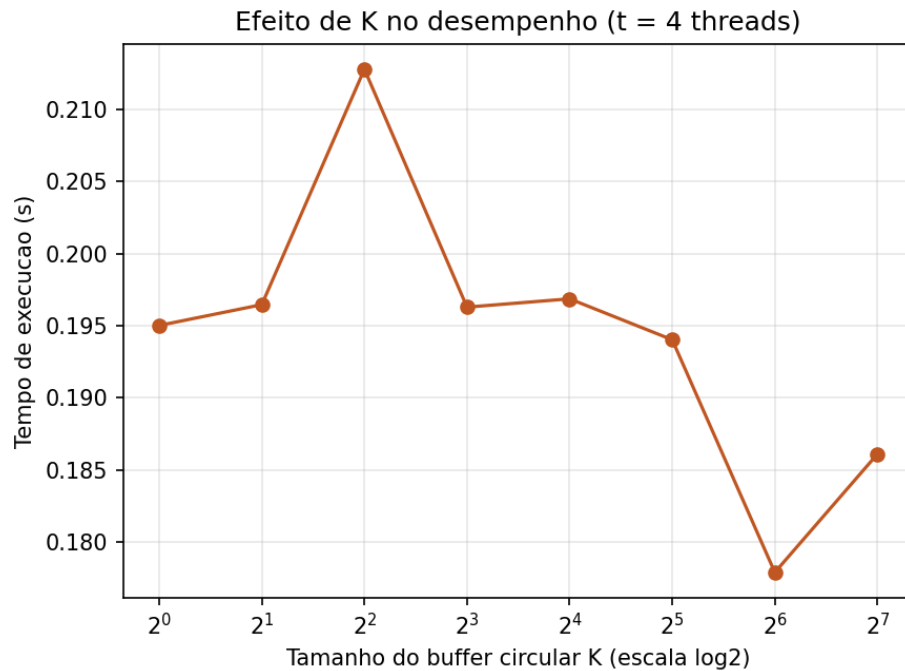


Figura 3 – Tempo de execução vs. tamanho do buffer circular K (t = 4 threads).

Diferentemente do esperado teoricamente, os tempos medidos variam pouco entre os diferentes valores de K (entre 0,178 s e 0,213 s), sem uma tendência monótona clara: o menor tempo ocorreu em K = 64 e o maior em K = 4, com os demais pontos próximos entre si. Isso sugere que, para este arquivo de teste e TAM\_BLOCO de 1 MB com apenas 4 threads produtoras, a consumidora consegue escrever os blocos praticamente no ritmo em que ficam prontos, de modo que o buffer raramente chega a ficar cheio mesmo com K pequeno — a variável dominante nas pequenas diferenças observadas passa a ser o ruído natural do sistema operacional (agendamento de threads, cache, outras cargas da máquina) e não o próprio tamanho do buffer. Esse resultado reforça que o efeito de K é mais pronunciado quando há descompasso real entre a velocidade de compressão (várias threads) e a velocidade de escrita (uma thread só) — cenário explorado de forma mais direta na discussão de K = 1, a seguir.

## 9 O QUE ACONTECE COM K = 1?

Com K = 1, o buffer circular possui uma única posição, o que significa que uma produtora só pode escrever o bloco i depois que a consumidora escreveu o bloco i – 1 no arquivo de saída. Isso elimina, na prática, o paralelismo do estágio de bufferização: mesmo com várias threads comprimindo blocos diferentes simultaneamente, apenas um bloco por vez pode estar em trânsito no buffer, e as demais threads produtoras ficam bloqueadas em pthread\_cond\_wait aguardando a liberação da única posição disponível.

No experimento realizado (Seção 8,  $t = 4$  threads),  $K = 1$  resultou em 0,195 s, valor muito próximo da maioria dos demais  $K$  testados (0,196 a 0,213 s), sem a penalidade acentuada que a teoria sugeriria. A explicação mais provável é que, com blocos de 1 MB, o tempo gasto comprimindo cada bloco é comparável ao tempo gasto escrevendo-o no disco, de modo que a consumidora praticamente não acumula atraso mesmo sendo obrigada a liberar a posição única antes de cada nova escrita. O efeito de  $K = 1$  tende a ficar mais evidente com blocos menores e mais threads produtoras, cenário em que a velocidade agregada de compressão supera com folga a velocidade de escrita sequencial — nessas condições,  $K = 1$  forçaria o sistema a se comportar como um pipeline essencialmente serial, com apenas a compressão do próximo bloco sobrepondo-se à escrita do bloco atual.

## 10 MUTEX GLOBAL VERSUS MUTEX POR POSIÇÃO DO BUFFER

A solução entregue (opzip.c) usa um único mutex global protegendo todas as  $K$  posições do buffer: qualquer operação de leitura ou escrita em qualquer posição, mesmo quando threads diferentes estão mexendo em posições diferentes, precisa adquirir esse mesmo lock, serializando o acesso ao buffer inteiro. Como alternativa, foi implementada uma variante experimental (opzip\_por\_posicao.c, incluída apenas para este experimento, e não como a solução principal) com um mutex e uma condition variable dedicados a cada posição do buffer, além de um mutex separado apenas para a variável de progresso (proximo\_a\_escrever). Nessa variante, threads que escrevem em posições diferentes do buffer não disputam o mesmo lock entre si.

Para medir o impacto da contenção, ambas as versões foram executadas com blocos pequenos (4 KB), de modo a maximizar a taxa de operações de lock por segundo. A Figura 4 apresenta os resultados.

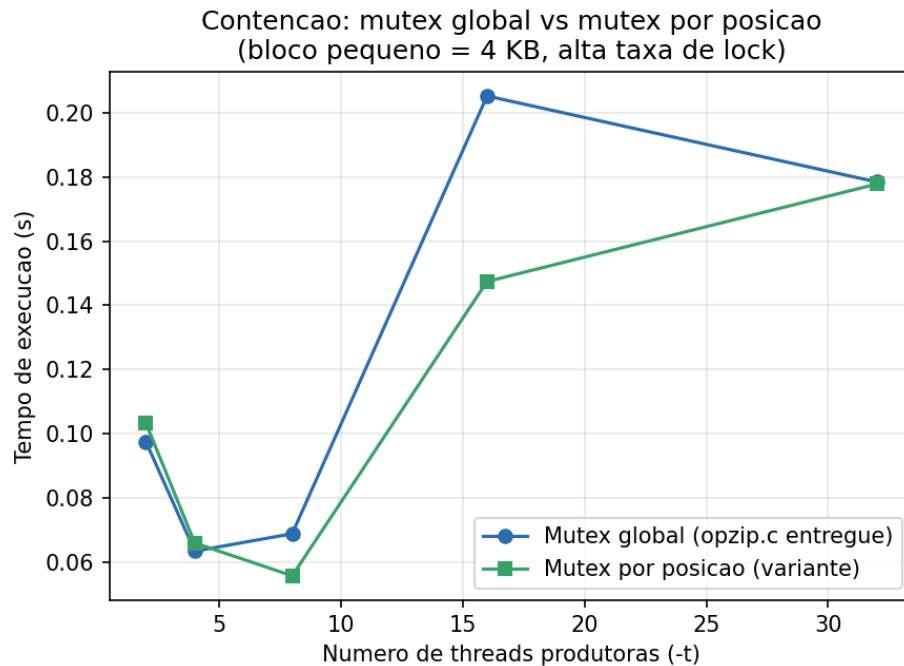


Figura 4 – Tempo de execução com mutex global vs. mutex por posição (bloco de 4 KB).

Os resultados mostram uma tendência favorável à versão com mutex por posição à medida que o número de threads cresce: com 8 threads, o mutex global levou 0,069 s contra 0,056 s da versão por posição (cerca de 19% mais rápida); com 16 threads, a diferença é ainda mais acentuada, 0,205 s contra 0,147 s (cerca de 28% mais rápida). Com poucas threads (2 e 4) as duas versões apresentam desempenho semelhante, e com 32 threads ambas convergem para um tempo próximo, provavelmente porque, nesse ponto, o gargalo deixa de ser a disputa pelo lock do buffer e passa a ser a saturação de núcleos disponíveis ou a própria escrita sequencial da consumidora. Esse padrão é consistente com a teoria: com o mutex global, toda operação no buffer fica serializada mesmo quando threads distintas mexem em posições distintas, criando um ponto de contenção artificial que se intensifica à medida que mais threads disputam o mesmo lock; a versão por posição elimina essa disputa desnecessária, permitindo que threads em posições diferentes progridam verdadeiramente em paralelo. O custo dessa abordagem é a complexidade adicional de manter  $K + 1$  pares de mutex e condition variable, em vez de apenas um, e a necessidade de manter dois níveis de sincronização coerentes entre si: a janela deslizante global e o estado individual de cada slot.

## 11 CONSIDERAÇÕES FINAIS

A implementação do opzip demonstrou, na prática, os principais conceitos trabalhados na disciplina de Sistemas Operacionais: uso de mutexes e condition variables para sincronização contínua entre múltiplas threads, distribuição dinâmica de trabalho para

balanceamento automático de carga, e o trade-off entre paralelismo e preservação de ordem na saída. O processo de desenvolvimento também evidenciou, de forma bastante concreta, os riscos clássicos da programação concorrente: um bug real de deadlock foi encontrado e corrigido durante os testes, decorrente de uma condição de sincronização aparentemente correta à primeira vista, mas insuficiente diante da distribuição dinâmica de blocos. Os experimentos de desempenho mostraram speedup próximo do linear até o número de threads compatível com os núcleos disponíveis na máquina de teste, com estagnação a partir daí devido ao gargalo estrutural da escrita sequencial única, e a comparação entre um mutex global e mutexes por posição do buffer confirmou que a granularidade da sincronização passa a importar à medida que o número de threads cresce, ainda que às custas de maior complexidade de implementação. Esses resultados reforçam que, em sistemas concorrentes, corretude e desempenho são preocupações interdependentes: uma sincronização excessivamente simples pode ser incorreta, enquanto uma sincronização excessivamente granular pode não compensar sua complexidade adicional dependendo da carga de trabalho.

## REFERÊNCIAS

ARPACI-DUSSEAU, Remzi H.; ARPACI-DUSSEAU, Andrea C. Operating Systems: Three Easy Pieces. Capítulos: Introduction to Threads; Threads API; Locks; Using Locks; Condition Variables. Arpaci-Dusseau Books, 2018.